# OOPs Concepts - Complete Guide



Class



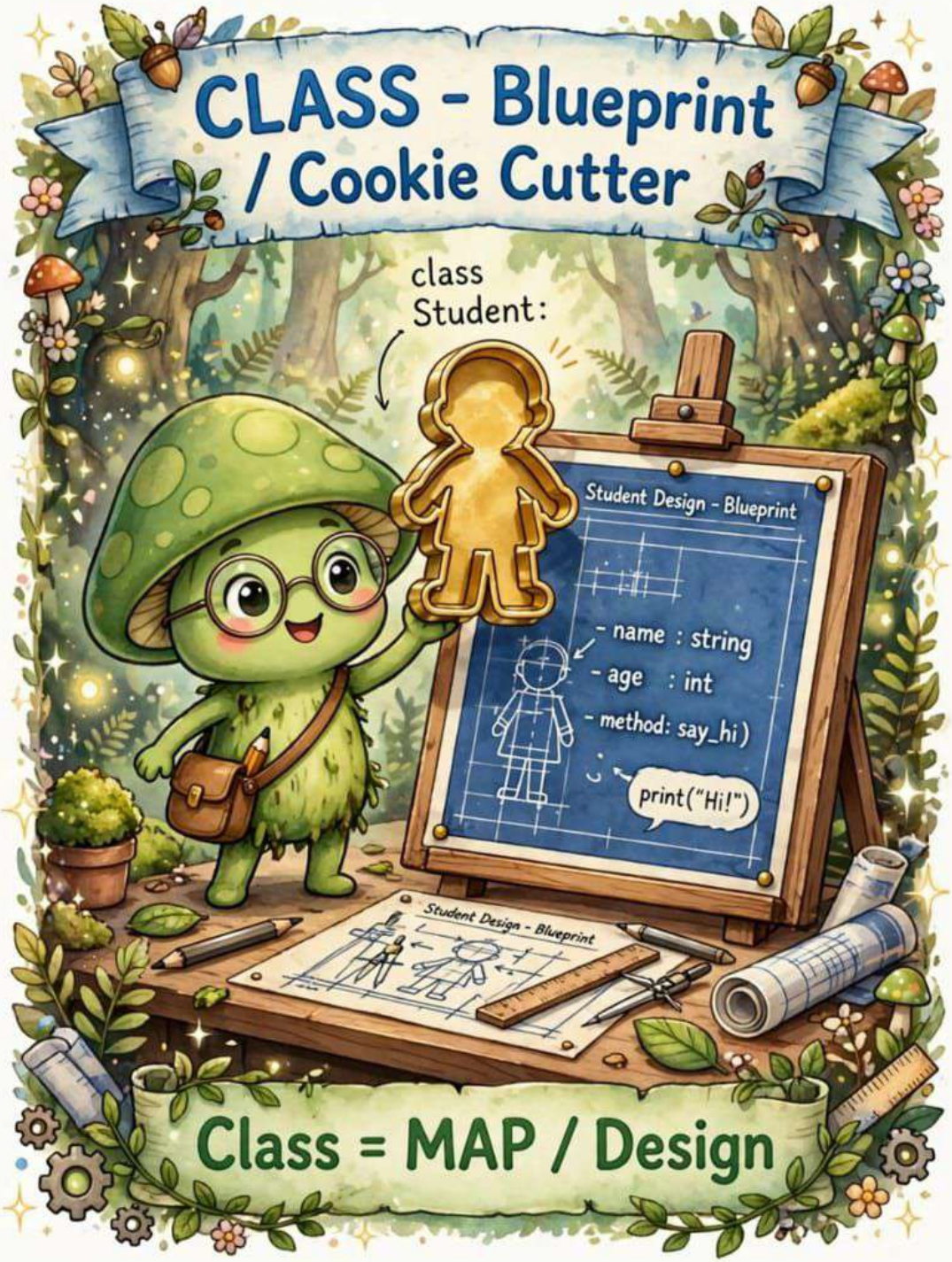
Inheritance

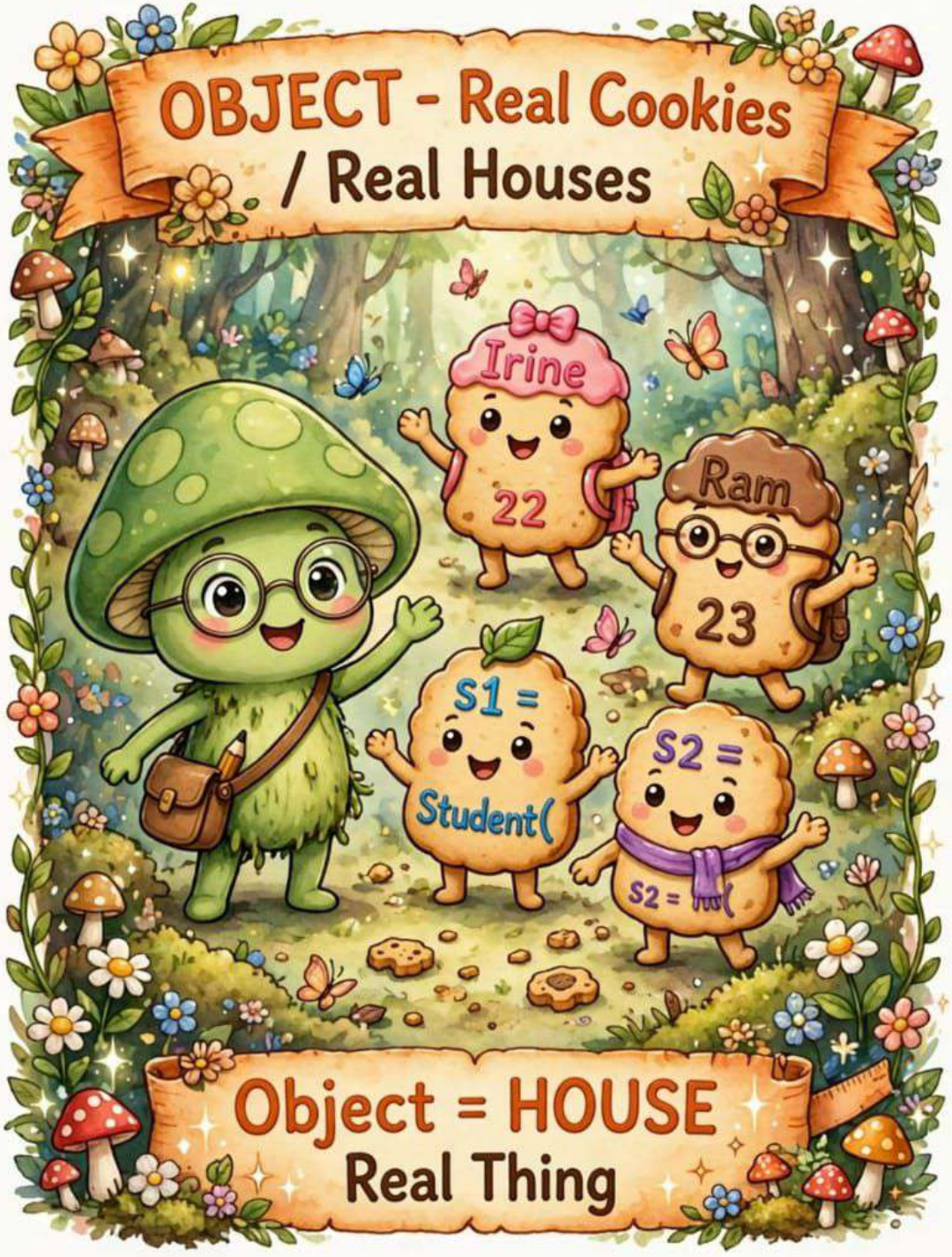


Polymorphism



By Irine Jeba | TechPanda Academy


CLASS - Blueprint
/ Cookie Cutter
class
Student:
Student Design - Blueprint
- name : string
- age : int
- method: say_hi)
print("Hi!")
Student Design - Blueprint
Class = MAP / Design


OBJECT - Real Cookies
/ Real Houses
Irine
22
Ram
23
S1 =
Student(
S2 =
S2 =
Object = HOUSE
Real Thing


__init__
- Constructor
Student()
__init=·: setup
object Born, setup everything
__del__
- Destructor
Student()
__del=·: cleanup
object Dies, cleanup & close

# Abstraction = Simple view | Car Analogy



# Encapsulation = Data Hiding + Safety

## Without Encapsulation - Unsafe

public variables – anyone can change!

```
self.balance = 100
  secret
  feelings
  100 coins...
```






Without = Open

## With Encapsulation - Safe

private variables
__balance – safe!

```
self.__balance =
getter() / setter() methods
→ controlled access!
```



With = Private __variable + getter/setter

Encapsulation keeps data safe inside the class — only you control access with getter & setter!

# Types of Inheritance - Family Tree

## 1. Single Inheritance

One parent mushroom giving house to one child mushroom.




## 2. Multilevel Inheritance




Grandparent → Parent): inherits → Child chain, passing house and properties down chain.

## 3. Multiple Inheritance




One child mushroom inherits from two parents, combining their properties.

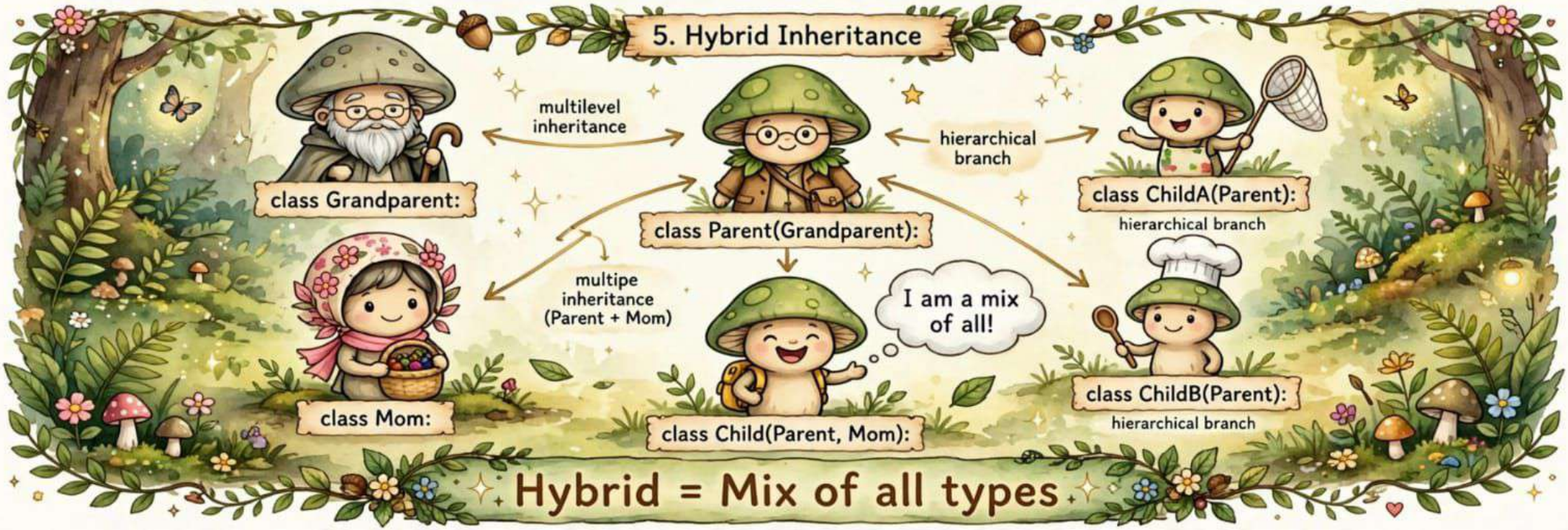
**Inheritance = Reuse Parent Code**

# Inheritance Part 2 - Advanced Types

## 4. Hierarchice Inheritance




## 5. Hybrid Inheritance




**Hybrid = Mix of all types**


Polymorphism = One Name, Many Forms
TOP PANEL:
Same method name speak()
def speak():
return "speak()"
def speak():
return "speak()"
def speak():
return "speak()"
Cat
Dog
Bird
Mushroom uses ONE remote →
Robot → dancing
Speaker → playing music
Light bulb glowing
TV → playing
One remote = Many actions
BOTTOM PANEL:
Different behavior
for animal in [Cat(), Dog(), Bird()]:
→ animal.speak() # same method name → different outputs!
Meow!
Woof!
Tweet!
Cat() → Meow
Dog() → Woof
Bird() → Tweet
KEY IDEA: Polymorphism lets the same function name work differently for different objects
— just like one remote controlling many devices!